German University in Cairo
Department of Computer Science
Assoc. Prof. Mervat Abu El-Kheir

Architecture of Massively Scalable Applications, Spring 2026
Task 1

Building, Versioning, and Containerizing a Spring Boot REST API

In this individual lab, you will develop a **Note-taking REST API** with **User management** using Spring Boot, manage its development with Git and GitHub using a multi-branch workflow that includes conflict resolution, and finally package and run it using Docker and Docker Compose. The task integrates the core concepts from the first three labs and challenges you to apply them in a slightly new context.

1 Project Setup & Initial Repository

Generate the Project

a) Go to `start.spring.io` and generate a Maven project with the following dependencies:

1. `Spring Web`
2. `Spring Boot DevTools`
3. `Rest Repositories`

b) Make sure to fill in the metadata in the following format (Java-valid and deterministic):

- ***Group:*** to be named this way ==> s55_xxxx.t_xx (e.g: s55_1234.t_08)
- ***Artifact:*** to be named this way ==> firstName_lastName (e.g: mohamed_ayman)
- ***Name:*** to be named this way ==> firstName_lastName (e.g: mohamed_ayman)
- ***Package Name:*** Make sure it will be formatted this way ==> s55_xxxx.t_xx.firstname_lastname (e.g: s55_8078.t_08.mohamed_ayman)

c) Open the project in IntelliJ IDEA and set the SDK to **JDK 25**.

d) Create the following packages inside your main source folder: `controllers`, `services`, `repositories`, `models`, `utils`

Initialise Git & Push to GitHub

a) Initialise a Git repository inside the project folder.

b) Stage all generated files and commit with the message:

```
Initial commit: project setup
```

c) Create a new **empty** repository on GitHub (no README, no license, no `.gitignore`) to be named as ***ID-Task-01-FirstName-LastName*** (e.g: 55-8078-Task-01-Mohamed-Ayman) .

d) ***Make sure that your repo is private one and not public***.

e) Link your local repository to the remote, rename the default branch to `main`, and push.

2 User Resource — `feat/users` Branch

You will implement the entire User resource on a dedicated feature branch.

Create the Branch

a) From `main`, create and switch to a new branch named `feat/users`.

User Model

a) In the `models` package, create a class `User` with the following fields:

1. `id : String` — generated as `UUID.randomUUID().toString()`
2. `username : String`
3. `email : String`
4. `createdAt : LocalDateTime` — set to `LocalDateTime.now()` when created

b) Provide getters and setters, an empty constructor, a constructor without `id`, and a full constructor. Make sure that all the constructors will initialize the createdAt as the current datetime when its not taken as an input and the empty constructor should also initialize the ID as a random UUID and the createdAt to be the current datetime.

Data Source

a) Download the users.json file and save it at ***src/main/resources*** using this link

User Repository Layer

a) Create `UserRepository` using the necessary annotations.

b) Declare a `private List<User> users` field.

c) In the constructor, read `users.json` using `ObjectMapper` and `ClassPathResource` (handle all exceptions). Use this code as the constructor for your repository:

```
private java.io.File jsonFile;

public UserRepository() {
    InputStream inputStream = getClass().getResourceAsStream("/users.json");

    if (inputStream == null) {
        throw new ResponseStatusException(
                HttpStatus.INTERNAL_SERVER_ERROR,
                "Unable to read users.json");
    }

    try {
        this.jsonFile = new java.io.File(getClass().getResource("/users.json").toURI());
    } catch (Exception e) {
        this.jsonFile = new java.io.File("/data/users.json");
    }

    ObjectMapper objectMapper = new ObjectMapper();
    this.users = objectMapper.readValue(inputStream, new TypeReference<List<User>>() {});
}
```

d) Make sure you have these imports to interact with the json files as some other imports may not work.

```
import tools.jackson.core.type.TypeReference;
import tools.jackson.databind.ObjectMapper;
```

e) Implement the following methods:

1. `List<User> findAll()` — returns the full list of users.
2. `Optional<User> findById(String id)` — returns the matching user wrapped in an `Optional`.
3. `Optional<User> findByUsername(String username)` — returns the matching user (not case senstive) wrapped in an `Optional`.
4. `User save(User user)` — generates a new UUID, adds the user to the list, and returns it. Make sure that the saved user is added to the json file by adding it to the list of users and writing this list to the json file. You may use this line for writing the list to the json file.

```
new ObjectMapper().writeValue(jsonFile, users);
```

5. `Optional<User> update(String id, User updated)` — finds the user by id, updates its fields, and returns it wrapped in an `Optional`. Don't forget to edit the json file using:

```
new ObjectMapper().writeValue(jsonFile, users);
```

6. `boolean deleteById(String id)` — removes the user from the list; returns `true` if found and deleted, `false` otherwise. Don't forget to edit the json file using:

```
new ObjectMapper().writeValue(jsonFile, users);
```

User Service Layer

a) Create `UserService` using the necessary annotation.

b) Inject `UserRepository` via constructor injection.

c) Implement the following methods:

1. `List<User> getAllUsers()` — delegates to the necessary function from the repository.
2. `User getUserById(String id)` — returns the user or throws a status `404` response if not found.
3. `User createUser(User user)` — calls `save()` and returns the persisted user.
4. `User updateUser(String id, User user)` — calls `update()` and returns the updated user, or throws a status `404` response if not found.
5. `void deleteUser(String id)` — calls `deleteById()`, throwing a status `404` response if not found.

User Controller Layer

a) Create `UserController` and make its enterypoint to all the routes within start with ***"/users"*** using the necessary annotations.

b) Inject `UserService`.

c) Implement the following endpoints:

1. `GET /users` — returns all users.
2. `GET /users/{id}` — returns a single user by id.
3. `POST /users` — accepts a JSON body without `id` and returns the created user.
4. `PUT /users/{id}` — accepts a new user as a request body to be updated and returns the modified user.
5. `DELETE /users/{id}` — removes the user and returns nothing.

6. `GET /users/search?username=...` — returns the user whose username matches the query string (case-insensitive), or `404` if none found.

The `GET /users/{id}/notes` endpoint will be added in a later step.

Update `application.properties`

a) Add the following line to `application.properties`:

```
app.description=Users and Notes API
```

Commit & Push

a) Stage all changes and commit with the message:

```
feat: add User model, repository, service and controller
```

b) Push `feat/users` to GitHub.

3 Note Resource — `feat/notes` Branch

Update `main`

a) Switch back to `main`.

b) Edit `application.properties` and add the following two lines:

```
server.port=8081
app.description=Note-taking REST API
```

c) Commit and push with the message:

```
config: set server port and app description on main
```

Create the Branch

a) From the updated `main`, create and switch to a new branch named `feat/notes`.

Note Model

a) In the `models` package, create a class `Note` with the following fields:

1. `id` : `String` — generated as `UUID.randomUUID().toString()`
2. `title` : `String`
3. `content` : `String`
4. `userId` : `String` — references the `id` of a `User`
5. `createdAt` : `LocalDateTime` — set to `LocalDateTime.now()` when created

b) Provide getters and setters, an empty constructor, a constructor without `id`, and a full constructor. Make sure that all the constructors will initialize the createdAt as the current datetime when its not taken as an input and the empty constructor should also initialize the ID as a random UUID and the createdAt to be the current datetime.

Data Source

a) Download the notes.json and save it at ***src/main/resources*** using this link:

Note Repository Layer

a) Create `NoteRepository` annotated with `@Repository`.

b) Declare a `private List<Note> notes` field.

c) In the constructor, read `notes.json` using `ObjectMapper` and `ClassPathResource` (handle all exceptions). Use this code as the constructor for your repository:

```
private java.io.File jsonFile;
public NoteRepository() {
        InputStream inputStream = getClass().getResourceAsStream("/notes.json");

        if (inputStream == null) {
            throw new ResponseStatusException(
                    HttpStatus.INTERNAL_SERVER_ERROR,
                    "Unable to read notes.json");
        }

        try {
            this.jsonFile = new java.io.File(getClass().getResource("/notes.json").toURI());
        } catch (Exception e) {
            this.jsonFile = new java.io.File("/data/notes.json");
        }

        ObjectMapper objectMapper = new ObjectMapper();
        this.notes = objectMapper.readValue(inputStream, new TypeReference<List<Note>>()
            {});
    }
```

d) Make sure you have these imports to interact with the json files as some other imports may not work.

```
import tools.jackson.core.type.TypeReference;
import tools.jackson.databind.ObjectMapper;
```

e) Implement the following methods:

1. `List<Note> findAll()` — returns the full list of notes.
2. `Optional<Note> findById(String id)` — returns the matching note wrapped in an `Optional`.
3. `List<Note> findByUserId(String userId)` — returns all notes belonging to the given user.
4. `Note save(Note note)` — generates a new UUID, adds the note to the list, and returns it. Make sure that the saved note is added to the json file by adding it to the list of notes and writing this list to the json file. You may use this line for writing the list to the json file.

   ```
   new ObjectMapper().writeValue(jsonFile, notes);
   ```

5. `Optional<Note> update(String id, Note updated)` — finds the note by id, updates its fields, and returns it wrapped in an `Optional`. Don't forget to edit the json file using:

   ```
   new ObjectMapper().writeValue(jsonFile, notes);
   ```

6. `boolean deleteById(String id)` — removes the note from the list; returns `true` if found and deleted, `false` otherwise. Don't forget to edit the json file using:

```
new ObjectMapper().writeValue(jsonFile, notes);
```

Note Service Layer

a) Create `NoteService` using the necessary annotation.

b) Inject `NoteRepository` via constructor injection.

c) Implement the following methods:

1. `List<Note> getAllNotes()` — delegates to necessary function in the repository layer.
2. `Note getNoteById(String id)` — returns the note or throws a status `404` response if not found.
3. `List<Note> getNotesByUserId(String userId)` — returns all notes for the given user or throws a status ***404*** response if not found.
4. `Note createNote(Note note)` — calls `save()` and returns the persisted note.
5. `Note updateNote(String id, Note note)` — calls `update()` and returns the updated note, or throws a status `404` response if not found.
6. `void deleteNote(String id)` — calls `deleteById()`, throwing a status `404` response if not found.

Note Controller Layer

a) Create `NoteController` and make its enterypoint to all the routes within start with ***"/notes"*** using the necessary annotations.

b) Inject `NoteService`.

c) Implement the following endpoints:

1. `GET /notes` — returns all notes.
2. `GET /notes/{id}` — returns a single note by id.
3. `POST /notes` — accepts a JSON body (including `userId`) and returns the created note.
4. `GET /notes/search?title=...` — returns notes whose title contains the query string (case-insensitive).
5. `PUT /notes/{id}` — accepts a new notes as a request body to be updated and returns the modified note
6. `DELETE /notes/{id}` — removes the note and returns nothing.

Commit & Push

a) Stage all changes and commit with the message:

```
feat: add Note model, repository, service and controller
```

b) Push `feat/notes` to GitHub.

4 Integration — Merging Branches & Resolving Conflicts

Merge `feat/notes` into `main`

a) Switch to `main` and merge `feat/notes`.

b) Push `main` to GitHub.

Merge `feat/users` into `main`

a) While still on `main`, merge `feat/users`.

b) A conflict will occur in `application.properties`. Git will mark it with those markers.

```
<<<<<<<, =======, >>>>>>>
```

c) Open `application.properties`, remove all conflict markers, and resolve the conflict by keeping the whole incoming changes from ***feat/user*** which is:

```
app.description=Users and Notes API
```

d) Stage the resolved file and complete the merge with the message:

```
Merge branch 'feat/users' into main: resolve app.description conflict
```

e) Inject `NoteService` into `UserController` and implement the following endpoint:

```
GET /users/{id}/notes
```

This endpoint should delegate to `NoteService.getNotesByUserId(id)`.

f) Commit with the message:

```
feat: wire GET /users/{id}/notes after notes merge
```

g) Push `main` to GitHub.

5 Advanced Git Workflows

Feature Branch — Add a Welcome Endpoint

a) From `main`, create and switch to a new branch named `feat/welcome`.

b) Add to your application.properties, the following environment variables:

```
USER_NAME=FirstName_LastName_Local
ID=55_xxxx
```

So that the environment variables would be for example: USER_NAME=Mohamed_Ayman_Local, ID=55_8078

c) Create `WelcomeController` in the `controllers` package with a `GET /welcome` endpoint that returns `'Hello <USER_NAME> <ID>, from Notes API'` and replace the USER_NAME and the ID with the environment variables values.

d) Commit the change with the message:

```
feat: add welcome endpoint
```

Parallel Change on `main`

a) Switch back to `main`.

b) Edit `application.properties` and append:

```
app.version=1.0.0
```

c) Commit and push with the message:

```
config: add application version property
```

Rebase the Feature Branch

a) Switch back to `feat/welcome` and rebase it on top of `main`.

b) Solve the conflicts by combining both changes and continue the rebasing (***keep the rebase/commit message as it is without changing***).

c) Force-push with lease the rebased branch.

d) Merge the ***feat/welcome*** branch into ***main*** By switching to the main branch first and push to the remote repo.

Accidentally Commit a Secret and Remove It

a) Switch back to `feat/welcome`, add a dummy API key to `application.properties`:

```
api.secret=SUPER-SECRET-KEY
```

b) Commit it — **do not push** — with the message:

```
add secret key
```

c) Revert this commit

d) Drop this commit using interactive rebase over the last 2 commits. If Git warns about an empty commit, ***skip it and do not allow it*** just like you learned in the lab.

e) Push the clean branch with force-with-lease.

6 Docker & Docker Compose — Containerization

Prepare the Application for Docker

a) Ensure all packages are installed:

b) Build the JAR

c) Test locally first before building the image

d) Verify `localhost:8080/notes`, `localhost:8080/users`, and `localhost:8080/welcome` all respond correctly.

e) Swtich back to ***main*** branch.

Dockerfile

a) Create a `Dockerfile` in the root folder of your project with the following instructions:

1. **Base Image** — use the same JDK used in the lab "eclipse-temurin:25.0.2_10-jdk".
2. **Working directory**.
3. **Copy** the JAR from the `target` folder into ***app.jar***.
4. **Copy** both JSON data files into a dedicated data "/data/notes.json" and "/data/users.json" directory inside the image:
5. **Environment variable**: set ***USER_NAME*** to **Docker_<FirstName_LastName>** (e.g: Docker_Mohamed_Ayman) and ***ID*** to Docker_55_xxxx (e.g: Docker_55_8078).
6. **Expose** the necessary port.
7. Specify the necessary commands to ***run the JAR*** inside the image.

b) Build the image

c) run the image and map the port ***9090*** to the exposed container port.

d) Verify your app is running by visiting ***localhost:9090/notes***, ***localhost:9090/users***, and ***localhost:9090/welcome***.

Docker Compose

a) Create `docker-compose.yaml` next to the Dockerfile.

b) Add your application as a service with the following configuration:

1. Build from the local `Dockerfile`.
2. Name the container as your ID_Container (e.g: 55-8078_container)
3. Map host port to container port `8080` make the host port as the second part of your ID (e.g: if your ID is 55-8078 then the port that you will use as host port will be 8078, making the mapping to be 8078:8080).
4. Set the `USER_NAME` environment variable to **Compose_<your_name>** and the the ***ID*** enviornment variable to ***Compose_ID*** (e.g: Compose_55_8078).

c) Start your services and Verify the service is responding correctly by visiting:

- `localhost:<secondIDPart>/welcome` (e.g: localhost:8078/welcome)
- `localhost:<secondIDPart>/notes` (e.g: localhost:8078/notes)
- `localhost:<secondIDPart>/users` (e.g: localhost:8078/users)
- `localhost:<secondIDPart>/users/{id}/notes` (e.g: localhost:8078/users/{id}/notes)

6.1 Last Required Commit

a) Stage your dockerfile and dockercompose file

b) Commit the files with the following message:

```
feat: add dockerfile and docker-compose
```

c) Push the commit to origin

7 Running the Automated Grader

Before submitting, you **must** run the automated grader against your project to verify your work. The grader checks your Git history, Docker image, Docker Compose setup, and runs JUnit tests against your code.

Make sure that you always run ***mvn clean install*** and then ***mvn package*** if you changed anything in the code and before running the grader again

Pull the Grader Image (one time)

```
docker pull abuelmagd/task1-grader:latest
```

Run the Grader

macOS / Linux (bash):

```
docker run --rm \
  -v /var/run/docker.sock:/var/run/docker.sock \
  -v "</absolute/path/to/your/repo>":/repo:ro \
  -e REPO_PATH=/repo \
  abuelmagd/task1-grader:latest
```

Windows (PowerShell):

```
docker run --rm ‘
  -v //var/run/docker.sock:/var/run/docker.sock ‘
  -v "<C:\Users\you\your-repo>":/repo:ro ‘
  -e REPO_PATH=/repo ‘
  abuelmagd/task1-grader:latest
```

Replace the path with the **absolute path** to your project folder — the one that contains `.git/`, `src/`, `Dockerfile`, and `docker-compose.yaml`.

Detailed Documentation

For full details on troubleshooting, and how to read the output and how to do partial runs (run with skipping options if you didn't finish the full task yet), refer to the documentation folder:

`https://jumpshare.com/folder/wtC84S5xN0uh6aCtcc9B`

The folder contains:

- `STUDENT_README.md` — How to run, all options, troubleshooting

Submission Requirements

A submission link will be provided later to submit your GitHub repo. Make sure to make it a private repo and add `Task-01-Scalable-2026` as a contributor.